Lecture 14
Polymorphism and Virtual Functions

Today's Lecture Contents

- Static Binding and Dynamic Binding
- Pointers and Derived Classes
- Derived class members invisible to base class pointers
- Base class members visible to base class pointers
- Polymorphism
- Virtual functions
- Virtual destructors

Static binding and dynamic binding

- Connecting a function call to a function body is called ***binding***.
  - Binding that takes place during compilation is called *static binding* (or e*arly binding*).

- By default, C++ matches a function call with the correct function definition at compile time. This is called *static binding*.

Static binding: Example

```
class Student {
public:
double calcTuition();
...
};
class GraduateStudent: public
Student {
public:
double calcTuition();
...
};
main() {
Student s;
s.calcTuition(); //calls
Student::calcTuition()
GraduateStudent gs;
gs.calcTuition(); //call
GraduateStudent::calcTuition()
}
```

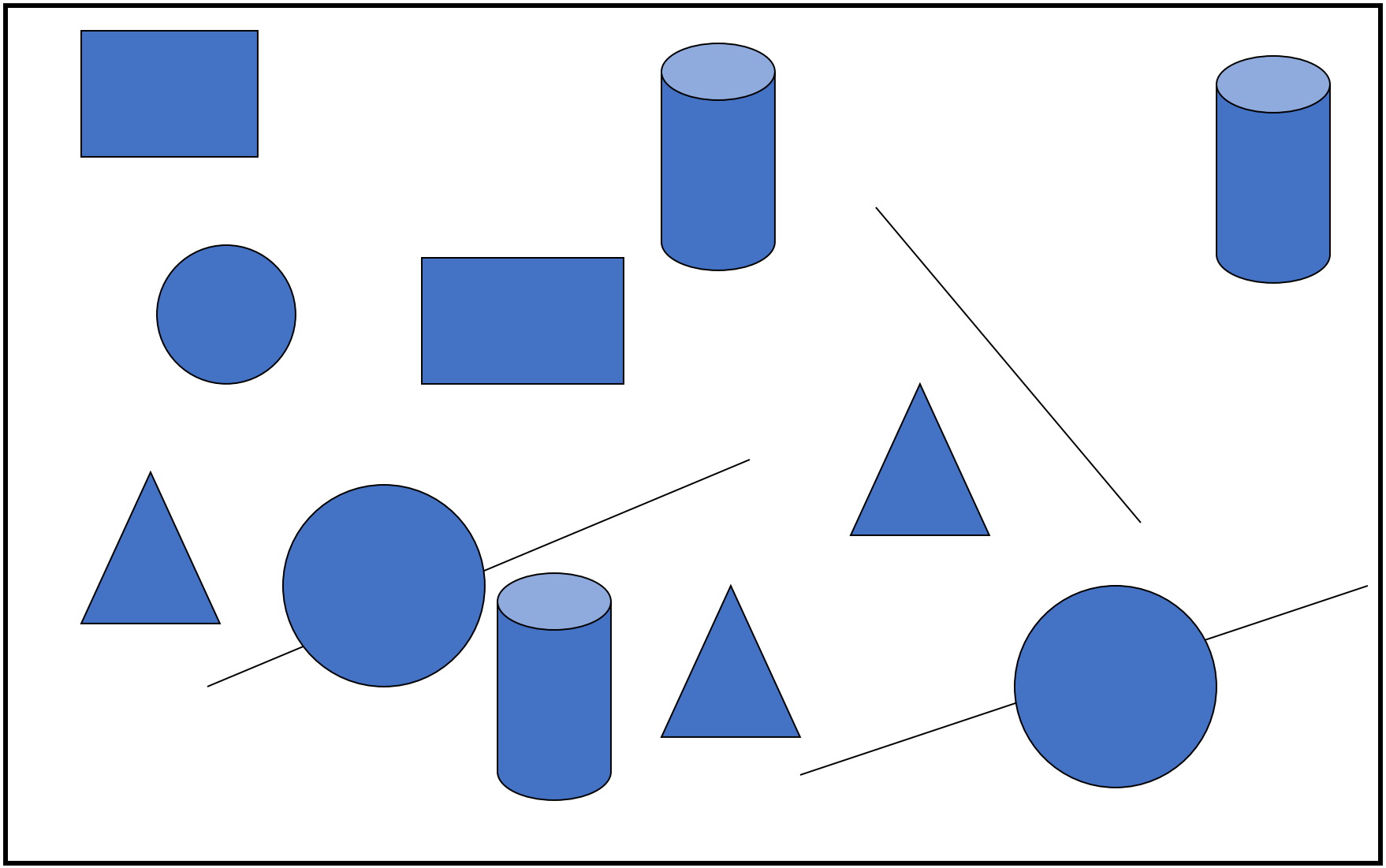
- calcTuition() is defined in base class Student.
- And then redefined in derived class GraduateStudent.
- At compile time, the compiler knows the type of the object,
- s in case of student::calcTuition() and
- gs in case of GraduateStudent::calcTuition().

Accessing class members using base class pointer pointing to derived class

```
class Student {
public:
double calcTuition();
};
class GraduateStudent: public
Student {
public:
double calcTuition();
};
main() {
Student s;
Student * sPtr = &s;
sPtr->calcTuition(); //calls
Student::calcTuition()
GraduateStudent gs;
sPtr = &gs;
sPtr->calcTuition(); //call
Student::calcTuition()
Student & sRef = gs;
sRef.calcTuition(); //call
Student::calcTuition()
}
```

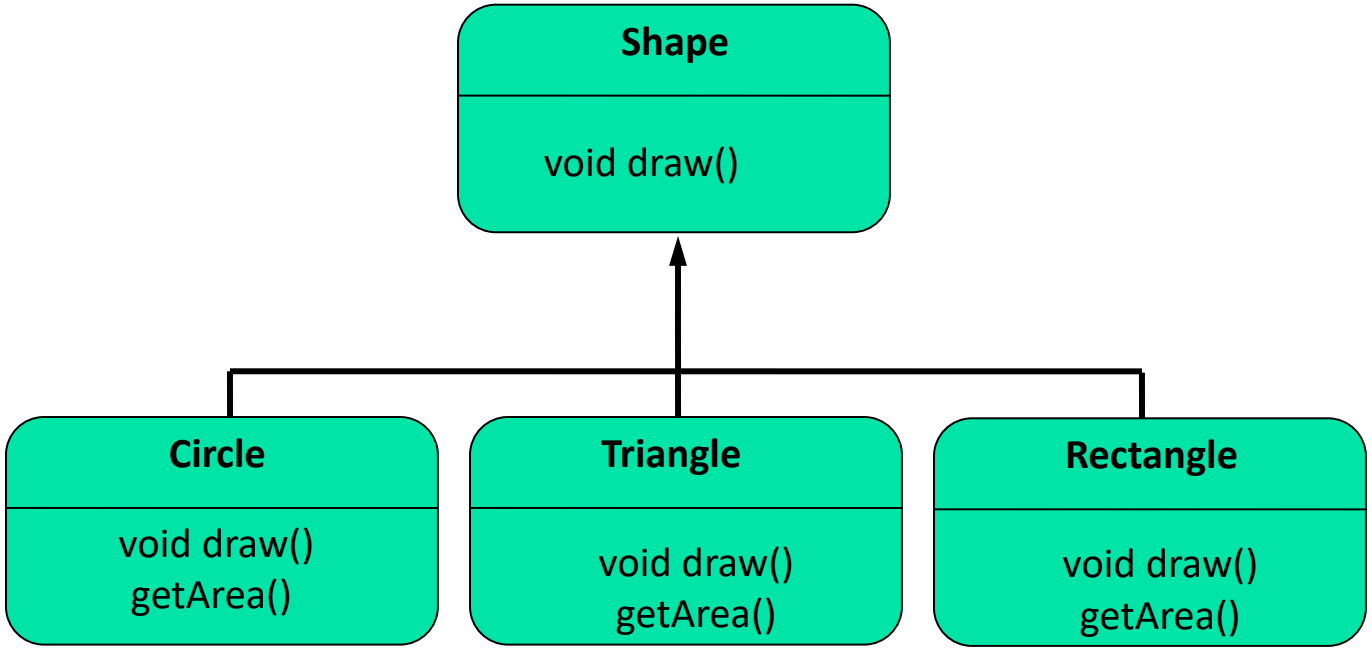
- C++ allows base class pointers to point to derived class objects.
- The call of the function calcTuition() is being set once by the compiler as the version defined in the base class during the compilation of the program.
- This is another example of static binding.

Graphics Drawing Software



Shape class hierarchy

- A frequently used example is a simple program to draw shapes on a computer screen.
- we could easily represent those shapes with an inheritance hierarchy or tree.
- The subclasses draw() function overrides the Shape draw() function.
- The subclasses class inherit everything that the Shape class has, including the draw() function, but Circle adds its own draw function, replacing the Shape draw() function.
- When a program calls the draw() function through a subclass object, it calls the subclass function instead of the inherited function.



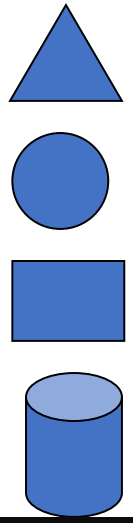
Graphics Drawing Software Classes

- **Shape**
  - Properties:- X-Y Coordinates, Length, Color
  - Actions:- Draw Function, Change Color Function,
  - Get Area Function.
  - Draw()
- **Circle**
  - Properties:- X-Y Coordinates, Radius, Color
  - Actions:- Draw Function, Change Color Function, Get Area Function.
  - Draw()
- **Rectangle**
  - Properties:- X-Y Coordinates, Width, Height, Color
  - Actions:- Draw Function, Change Color Function, Get Area Function.
  - Draw()
- **Cylinder**
  - Properties:- X-Y Coordinates, Radius, Height, Color
  - Actions:- Draw Function, Change Color Function, Get Area Function.
  - Draw()
- **Triangle**
  - Properties:- X-Y Coordinates, Length, Width, Color
  - Actions:- Draw Function, Change Color Function, Get Area Function.
  - Draw()

Shape class pointers to Derived Classes

- Trying to access derived class members using shape class pointer.
- The call of the function area() is being set once by the compiler as the version defined in the Shape class during the compilation of the program.
- Therefore, regardless of which object the pointer points to it always executes the Shape class function.

```
int main()    {
  shape *shapeptr;
  shapeptr = new Triangle (3, 4, 5, 19 );
  shapeptr->draw ();
  shapeptr->GetArea ( );

  shapeptr = new Circle (3, 4, 5 );
  shapeptr->draw ();
  shapeptr->GetArea ( );

  shapeptr = new Rectangle ( 3, 4, 10 , 20 );
  shapeptr->draw ();
  shapeptr->GetArea ( );

  shapeptr = new Cylinder ( 3, 4, 5, 10 );
  shapeptr->draw ();
  shapeptr->GetArea ( );
}
```



```
shape Drawing code

shape Area

shape Drawing code

shape Area
```

- But what if the exact class of the object can't be determined at compile time?

Revisiting the Student class

```
class Student {
public:
double calcTuition();
...
};
class GraduateStudent: public Student
{
public:
double calcTuition();
...
};
void main() {
Student s;
GraduateStudent gs;
Func(s);
Func(gs);
}
```

- But what if the exact class of the object can't be determined at compile time?
- Instead of calling calcTuition() directly, the call is now made through an intermediate function, func(Student &x).
- Depending on how func(Student &x) is called, x can be a Student or a GraduateStudent.
- You would like x.calcTuition() to call Student::calcTuition() when x is a Student
- but call GraduateStudent::calcTuition() when x is a GraduateStudent.

```
void func(Student &x) {
x.calcTuition(); //to which calcTuition()
does this refer?
} //(answer: base class Student::calcTuition)
```

Polymorphism and Virtual Functions

- It is possible to determine type of argument at runtime instead of the compile time. Delaying the binding to the runtime is called **Dynamic binding**.
- we can declare a function with the keyword **virtual** and override by the derived classes. if we want the compiler to use dynamic binding for that specific function.
- The capability to decide at run time which of several overloaded member functions to call based on the *actual type* is called **polymorphism**.
- ***Poly*** means many and ***morph*** means form.
- Terms Dynamic Binding and Polymorphism are used interchangeably.

Revisiting the Student Class

```
class Student {
public:
virtual double calcTuition();
//...other stuff
};
class GraduateStudent: public Student
{
public:
virtual double calcTuition();
//...other stuff
};
void main() {
Student s;
GraduateStudent gs;
Func(s);
Func(gs);
}
```

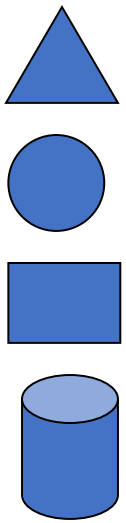
- The virtual keyword indicates to the compiler that it should choose the appropriate definition of calcTuition() not by the type of reference, but by the type of object that the reference refers to.
- Therefore, a ***virtual function*** is a member function you may redefine for other derived classes, and can ensure that the compiler will call the redefined virtual function for an object of the corresponding derived class, even if you call that function with a pointer or reference to a base class of the object.

```
void func(Student &x) {
x.calcTuition(); //to which calcTuition()
does this refer?
} //(answer: depends on the type of the
argument passed)
```

Pointers to Derived Classes (re-visiting shape class)

- Declaring draw() and GetArea() **virtual** in shape class, they can be accessed based on the type of the object and NOT on the type of the handle.

```
int main()    {
  shape *shapeptr;

  shapeptr = new Circle (3, 4, 5 );
  shapeptr->draw ();
  shapeptr->GetArea ( );

  shapeptr = new Rectangle ( 3, 4, 10 , 20 );
  shapeptr->draw ();
  shapeptr->GetArea ( );

}
```



```
Circle drawing code
Circle area code
Rectangle drawing code
Rectangle area code
```

Processing different types of Objects in homogeneous way

```
int main() {
   shape *shapeptr[4];
   shapeptr[0] = new Triangle (3, 4, 5, 19 );
   shapeptr[1] = new Circle (3, 4, 5 );
   shapeptr[2] = new Rectangle ( 3, 4, 10 , 20 );
   shapeptr[3] = new Cylinder ( 3, 4, 5, 10 );
```

```
for ( int i = 0; i < 4; i ++ )
   {          shapeptr[i]->draw ();
              shapeptr[i]->GetArea ( );
   }
```

```
Circle drawing code
Circle area code
Rectangle drawing code
Rectangle area code
```

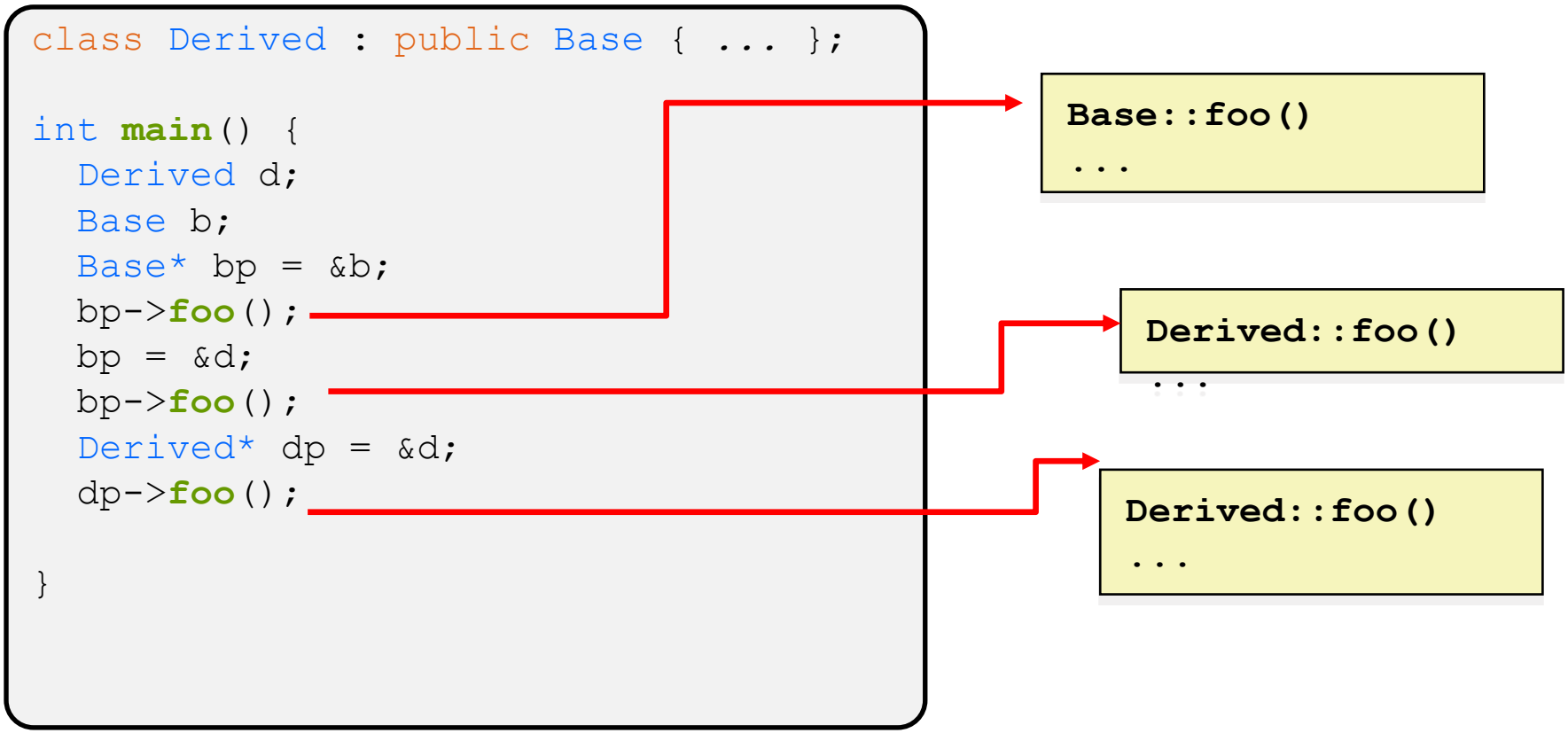
Processing different types of Objects in homogeneous way

```
void drawAll(shape* figure[], int n) {
            for(int i=0; i<n; ++i) {
   figure[i]->draw();
 figure[i]->getArea();
  }
}

int main() {
   shape *shapeptr[4];
   shapeptr[0] = new Triangle (3, 4, 5, 19 );
   shapeptr[1] = new Circle (3, 4, 5 );
   shapeptr[2] = new Rectangle ( 3, 4, 10 , 20 );
   shapeptr[3] = new Cylinder ( 3, 4, 5, 10 );

drawAll(shapeptr, 4);
}
```

```
Circle drawing code
Circle area code
Rectangle drawing code
Rectangle area code
```

Conclusion

- If foo() is a virtual member function of Base
  - And foo() is redefined in Derived class.



Summary – Based and Derived Class Pointers

- Base-class pointer pointing to base-class object
  - Straightforward
- Derived-class pointer pointing to derived-class object
  - Straightforward
- Base-class pointer pointing to derived-class object
  - Safe
  - **Can access virtual methods of derived class**
- Derived-class pointer pointing to base-class object
  - Compilation error

Destructor in programs using polymorphism

Destructors in programs using polymorphism

- Programs using polymorphism uses base class pointer pointing to the derived class object.
- If the classes have destructors, then these destructors are statically bound based on the type of the pointer and the destructor for the actual object pointed to by the pointer will not be called when the object is deallocated. For example:

```
B *bptr = new D;

delete bptr;
```

- Now after `delete bptr` is executed, derived class object will go out of scope, however, its destructor will not be called, only destructor of the base class will be called base on the fact that bptr is of type base class.

```
class B {
public: B() {
 cout<< "Base Cstor" << endl;  }
~B() {
cout<< "~Base Dstor" << endl; }
 };
```

```
class D : public B  {
public: D()  {
cout<< "Derived Cstor" << endl;  }
~D() {
cout<< "Derived Dstor" << endl;  }
};
```

Example 1: derived class destructor not called

```
class B {
public: B() {
 cout<< “Base Cstor” << endl;
}

~B() {
cout<< “~Base Dstor” << endl;
}
 };

 class D : public B
{
 public: D()
{
 cout<< “Derived Cstor” << endl;

 }
 ~D() { cout<< “Derived Dstor” << endl;
 }
 };
int main() {
B *b = new D;
delete b; }
```

Output:

```
Base Cstor
Derived Cstor
Derived Dstor      // Not Called
Base Dstor
```

- Deleting a base pointer only calls destructor for the base class not the destructor for the derived class.
- The derived parts of the object never destroyed.
- The object is partially destroyed.

Example 2: derived class destructor not called

- Deleting a base pointer only calls destructor for the base class not the destructor for the derived class.

```
Base::~Base()

Base::~Base()
```

```
class Base {
 int* x;
 public:
  Base() { x = new int; }
  ~Base() { delete x; }
};

class Der1 : public Base {
 int* y;
 public:
  Der1() { y = new int; }
  ~Der1() { delete y; }
};

void main() {
  Base* b0ptr = new Base;
  Base* b1ptr = new Der1;

  delete b0ptr;// deletes x
  delete b1ptr;//only deletes x, leaks y
}
```

Virtual Destructors

- Whenever we have virtual functions, we should always declare the destructor to be virtual.
- Making the destructor virtual will ensure that the correct destructor will be called for an object when pointers to the base classes are used.
- If the destructor is not declared to be virtual, then the destructor is statically bound based on the type of pointer and the destructor for the actual object pointed to will not be called when the object is deallocated.
- For example:

```
~base (); //non-virtual base class destructor.
```

Now if the derived class object goes out of scope, derived class destructor will not be called, only base class destructor will be called.

Virtual Destructors

- You need a virtual destructor when someone will delete a derived class object via a base class pointer, i.e.,

```
Virtual ~base();    //derived class destructor automatically becomes virtual.
```

- Example:

```
Base* p = new Derived;

delete p; //since destructor is virtual, first derived then base destructor is called
```

- A simple rule is to have a virtual destructor whenever your class has at least one virtual function
- Note: if the base class destructor is virtual, the derived classes destructors will be virtual automatically

Example 1 (revisiting): virtual destructor

```
class B {
public: B() {
 cout<< “Base Cstor” << endl;
}

Virtual ~B() {
cout<< “~Base Dstor” << endl;
}
 };

 class D : public B
{
 public: D()
{
 cout<< “Derived Cstor” << endl;

 }
 ~D() { cout<< “Derived Dstor” << endl;
 }
 };
int main() {
B *b = new D;
delete b; }
```

Output:

Base Cstor
Derived Cstor
Derived Dstor
Base Dstor

- if we declare the base class destructor as virtual, this makes all the derived class destructors virtual as well
- Deleting a base pointer calls destructor for the base class as well as destructor for the derived class.
- The derived parts of the object are also destroyed.

Example 2: virtual destructor

- Good habit to always define a dtor as virtual
  - Empty body if there's no work to do

```
Base::~Base()

Der1::~Der1()

Base::~Base()
```

```
class Base {
 public:
  Base() { x = new int; }
  virtual ~Base() { delete x; }
  int* x;
};

class Der1 : public Base {
 public:
  Der1() { y = new int; }
  ~Der1() { delete y; }
  int* y;
};

void main() {
  Base* b0ptr = new Base;
  Base* b1ptr = new Der1;

  delete b0ptr;// deletes x
  delete b1ptr;//deletes both x and y
}
```

upcasting

- Converting a value from one data type to another is called ***typecasting or casting*** for short.
- Casting an object from one class type to another makes sense when the original class type is a subclass of the destination type.
- For example, if Circle is a subclass of Shape, ***a Circle "is a" Shape***, so it makes sense to think of a Circle as a Shape or to convert an instance of Circle to an instance of Shape.

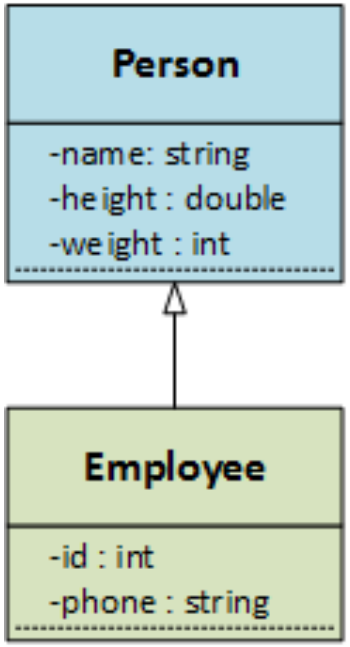
```
Shape* sptr = new Circle; //upcasting
```

- when we cast objects, we always do so in the context of inheritance. If we arrange the inheritance hierarchy with each superclass above its subclasses, casting from a subclass to its superclass always takes place upwards in an inheritance hierarchy.

Upcasting: Another example

```
Person* ceo = new Employee;
```

- This statement, instantiates an Employee object but upcasts it to a Person.
- Programs rarely perform an upcast operation with a simple assignment statement such as above
- Programs most often perform an upcast when they call a function, passing a subclass object to a superclass parameter



Downcasting

- It is also possible and sometimes necessary to cast "downwards" in an inheritance hierarchy.

```
Shape* sptr = new Circle; //upcasting
```

- This statement instantiates a circle object but upcasts it to a Shape.
- However, if the the circle object has a circle specific member function such as ***showradius()*** then such subclass specific member functions is not unreachable from the superclass pointer. For example, the below is not possible:

sptr->showradius(); //does not work

- The only way to execute the showradius() specified in the circle class is ***to downcast sptr to a circle***.
- Note that upcasting is done by the compiler automatically.
- Downcasting requires ***explicit cast operation***, i.e., compiler makes us explicitly perform the downcast.
-

Downcasting using `static_cast` operator

- Converting base class pointer back into derived class pointer
- dynamic_cast represents an attempt to cast the pointer to the class named in angle brackets.

```
Class shape {
public:
        virtual draw() {}};
Class circle: public shape {
        float radius;
Public:
        circle(float r):radius(r) {}
        virtual void draw(){ cout<<"Draw circle: "<<endl;}
        void showradius() { cout<<radius<<endl; }  };
int main(){
Shape *sptr = new circle(5.5);
Sptr->draw();
//we cannot call getradius() using sptr.
//we have to downcast sptr back to circle
static_cast<circle *>(sptr)->showradius();
Cout<<"radius is: "<<Cptr->getradius()<<endl; }
```